

GitHub for Data Engineering

A FREE practical learning guide for Git, GitHub, CI/CD, and real data engineering project integration

Prepared for: Data Engineers, Freshers, Working Professionals, and Job Seekers

Brand: Datageeks

Focus: Real project usage - not just commands

This guide is written in a practical way: what GitHub means inside actual data engineering teams, how CI/CD fits into pipelines, and how to learn it using free resources.

Table of Contents

1. Why GitHub matters in Data Engineering
2. Git vs GitHub - simple difference
3. Daily GitHub workflow used in teams
4. Important Git commands cheat sheet
5. Branching, Pull Requests, and code reviews
6. GitHub Actions and CI/CD basics
7. Data Engineering project integration example
8. Sample GitHub Actions workflow YAML
9. Common mistakes to avoid
10. 14-day learning roadmap
11. Free resources and source links

Datageeks

1. Why GitHub matters in Data Engineering

Many people think GitHub is only for software developers. In real data engineering jobs, it is equally important. Your pipelines, SQL scripts, PySpark jobs, Databricks notebooks, dbt models, Terraform code, ADF templates, and deployment files all need proper version control.

The practical reason is simple: production data pipelines cannot depend on files saved locally on someone's laptop. Teams need a repeatable way to review, test, approve, and deploy code safely.

Area	How GitHub helps Data Engineers
Version control	Track every change in SQL, Python, PySpark, YAML, and config files.
Team collaboration	Multiple engineers can work using branches and pull requests.
Quality control	Code review and automated checks reduce accidental production issues.
Deployment	CI/CD workflows move changes from dev to QA/prod in a controlled way.
Auditability	You can see who changed what, when, and why.

2. Git vs GitHub - simple difference

Git is the version control tool. GitHub is the cloud platform where teams host Git repositories, collaborate through pull requests, and automate CI/CD using GitHub Actions.

GitHub Branching + Pull Request Workflow

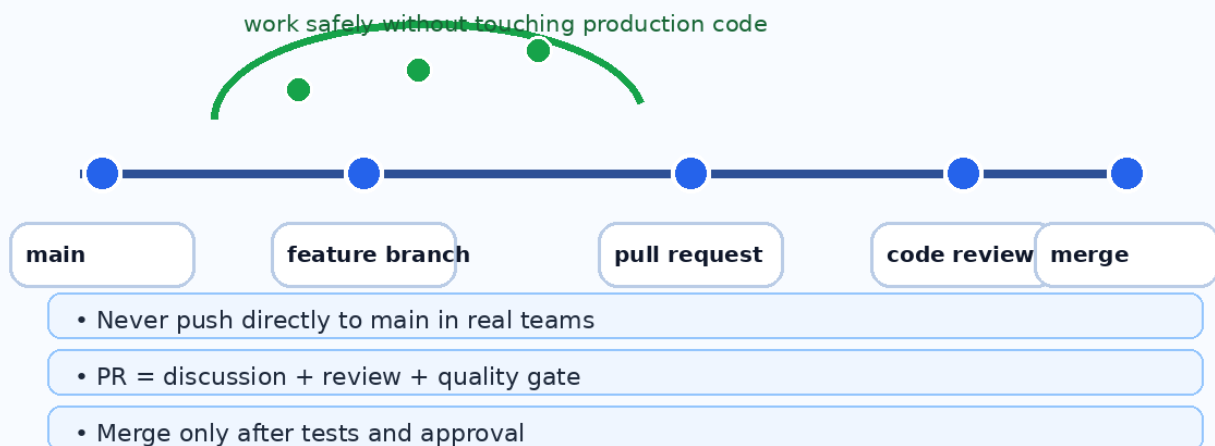


Image note: Educational workflow diagram created for this guide, based on standard GitHub branch and pull request concepts from GitHub and Atlassian learning resources.

3. Daily GitHub workflow used in teams

Step	What you do	Real project example
1	Clone the repository	Get the project code on your laptop or VM.
2	Create a feature branch	Work on a new pipeline or SQL change safely.
3	Make changes locally	Update PySpark logic, SQL model, YAML config, or tests.
4	Commit changes	Save a clean checkpoint with a meaningful message.
5	Push branch	Send your branch to GitHub.
6	Raise Pull Request	Ask team members to review your change.
7	CI/CD checks run	Linting, tests, build, and deployment validation happen.
8	Merge after approval	Change goes to main branch and deployment flow starts.

4. Important Git commands cheat sheet

Command	Purpose
git --version	Check whether Git is installed.
git config --global user.name "Your Name"	Set your Git username.
git config --global user.email "you@email.com"	Set your Git email.
git clone	Copy a remote repository to your local system.
git status	See changed, staged, and untracked files.
git checkout -b feature/my-change	Create and switch to a new branch.
git add .	Stage all changed files.
git commit -m "message"	Commit staged changes with a message.
git pull origin main	Bring latest main branch changes locally.
git push origin feature/my-change	Push your branch to GitHub.
git log --oneline	View compact commit history.
git diff	See file-level changes before commit.
git merge main	Merge main branch into your current branch.
git stash	Temporarily save unfinished work.
git revert	Safely undo a previous commit.

5. Branching, Pull Requests, and code reviews

In many companies, the main branch is protected. You do not directly push code to main. You create a branch, push your work, raise a Pull Request, wait for review, fix comments, and then merge. This is not formality - this is how production mistakes are reduced.

Good branch names: feature/customer-pipeline, bugfix/null-handling, hotfix/prod-alert-fix, chore/readme-update

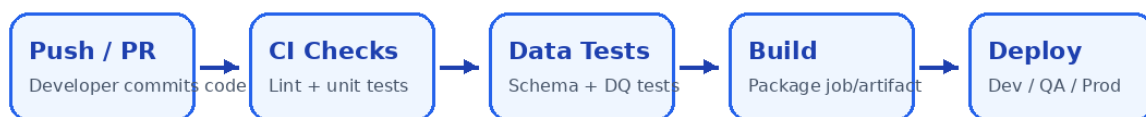
Good commit messages: Add DQ checks for customer feed, Fix date parsing in sales pipeline, Update ADF deployment parameter file

Bad habit to avoid: committing everything as "final", "new changes", "updated", or "test". In real teams, this creates confusion during production debugging.

6. GitHub Actions and CI/CD basics

GitHub Actions lets you automate workflows inside the repository. A workflow is defined using YAML files inside `.github/workflows/`. For data engineering, this can run code checks, unit tests, data quality checks, packaging, and deployment steps.

Simple GitHub Actions CI/CD for Data Engineering



Real data engineering examples:

- Run SQLFluff/ruff checks before merge
- Run pytest for transformation logic
- Validate dbt models or data-quality rules
- Deploy Databricks/ADF/dbt code only after approval

Image note: Educational CI/CD diagram created for this guide, aligned with GitHub Actions workflow concepts from GitHub Docs and Microsoft Learn.

7. Data Engineering project integration example

Use this project idea to practice GitHub like a real Data Engineer: build a small batch pipeline that reads raw sales CSV data, validates schema, transforms it using Python/PySpark, writes curated output, and runs tests automatically through GitHub Actions.

GitHub Integrated Data Engineering Project



Repository structure example

```
data-eng-github-project/
├── src/           # pipeline code
├── sql/          # SQL scripts or dbt models
├── tests/        # unit + data-quality tests
├── configs/      # env configs
├── .github/workflows/ # CI/CD YAML files
├── requirements.txt
└── README.md
```

Recommended project name: sales-data-pipeline-github-cicd

What to include: README, source code, sample input, transformation logic, tests, requirements.txt, CI/CD workflow YAML, and screenshots of successful workflow runs.

8. Sample GitHub Actions workflow YAML

```
name: data-pipeline-ci

on:
  pull_request:
    branches: [ main ]
  push:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
```

```
- name: Install dependencies
run: |
python -m pip install --upgrade pip
pip install -r requirements.txt

- name: Run lint checks
run: |
python -m compileall src

- name: Run unit tests
run: |
pytest tests/ -v
```

Start simple. First automate lint and tests. Later add dbt checks, SQL validation, Docker build, Databricks deployment, Terraform plan, or cloud-specific deployment steps.

Datageeks

9. Common mistakes to avoid

Mistake	Better approach
Learning only commands	Practice using a real mini project and PR flow.
Pushing directly to main	Use branches and pull requests.
No README	Document purpose, setup, run steps, and folder structure.
No tests	At least add unit tests for transformation logic.
No .gitignore	Do not commit virtual environments, logs, secrets, or large outputs.
Hardcoding secrets	Use GitHub Secrets or cloud secret managers.
Poor commit messages	Write messages that explain actual change intent.
Ignoring failed CI	Treat workflow failures like real project blockers.

10. 14-day learning roadmap

Days	Focus	Output
1-2	Git basics: init, clone, add, commit, status	Local repo with 5 meaningful commits
3-4	Branching and merging	Feature branch merged locally
5-6	GitHub repo, push, pull, README	Public/private repo with clean README
7-8	Pull Request and code review flow	PR created, reviewed, and merged
9-10	GitHub Actions basics	Workflow runs on PR
11-12	Data pipeline project setup	src, tests, sample data, requirements
13	CI checks for project	pytest or validation checks run automatically
14	Polish portfolio	Screenshot, README, and learning notes added

11. FREE resources and source links

Use these free resources. Prefer official documentation first, then tutorials for practice.

Resource	Use it for	Link
GitHub Skills	Hands-on GitHub learning through real GitHub features.	https://skills.github.com/
GitHub Skills Repository	Interactive exercises for beginners and experts.	https://github.com/skills
GitHub Actions Documentation	Official documentation for workflows, jobs, runners, and CI/CD.	https://docs.github.com/actions
GitHub Actions Feature Page	Overview of GitHub Actions capabilities.	https://github.com/features/actions
Microsoft Learn - Introduction to GitHub Actions	Beginner-friendly learning module for workflows, events, jobs, and runners.	https://learn.microsoft.com/en-us/training/modules/introduction-to-github-actions/
Microsoft Learn - GitHub Actions Learning Path	Structured learning path for automating software development workflows.	https://learn.microsoft.com/en-us/training/paths/github-actions/
Atlassian Git Tutorials	Strong Git basics: branching, pull requests, merging, and core commands.	https://www.atlassian.com/git/tutorials
Atlassian Basic Git Commands	Good quick reference for basic Git commands.	https://www.atlassian.com/git/glossary
Atlassian git init tutorial	Repository setup and init/clone workflow basics.	https://www.atlassian.com/git/tutorials/setting-up-a-repository
Git official docs	Core Git reference and book-style learning.	https://git-scm.com/doc

Image and concept source note: GitHub/Git icon assets available locally from open-source packages in the generation environment. Workflow diagrams in this guide are original educational diagrams created for Datageeks and aligned with concepts from GitHub Docs, GitHub Skills, Microsoft Learn, Atlassian Git Tutorials, and Git documentation.

Final note: If you are preparing for data engineering jobs, do not keep GitHub as a side topic. Build one project, use branches, raise PRs, add CI/CD, and make your work visible.